

LC-30: Substring with Concatenation of All Words

1. Problem Overview

Basically, we are given a string **S** where we need to look for specific substrings formed by concatenating elements from a vector of strings.

Example: Let the vector be ["ab", "bc", "cd"]. The substrings to look for are their concatenations:

abbccd | abcd bc | bcabcd | etc.

Key Properties:

- It is guaranteed that the word length in the vector is the same for all words.
- We need to return the start indices of these valid substrings.
- Since word length is fixed and all vector elements must participate in the concatenation exactly once, the total substring length (Window Size) is:

$$\text{Window Size} = (\text{Word Length}) \times (\text{Number of Words})$$

2. Key Observations

Observation 2 (Most Important):

Unlike other sliding window questions where individual characters matter, here a **group of characters (a word)** is treated as a single unit.

Consider the example: `S = "barfoothefoobarman", words = ["foo", "bar"]`

Traversal Strategy	Units Evaluated	Status
Character-by-character	barfoo → arfoot → rfootm	✗ Wrong Traversal
Word-by-word (Chunks)	bar foo → foo the → the foo	✓ Correct Traversal

This approach is necessary because the *order of characters within a word matters*, and extracting fixed-length chunks guarantees we are evaluating complete words.

3. Algorithm & Logic Steps

S1: Initialize an `unordered_map<string, int> mp1`; to store the frequency of each word required from the words vector.

S2: Iterate over every possible offset. Since words are processed in chunks of `wordlen`, we need outer iterations to check all alignments. Set a left and a right pointer that jump by `wordlengths`.

```
for(int i = 0; i < wordlen; i++)
```

S3: Inside the offset loop, initialize a tracking map `mp2`, the left boundary `l = i`, and a count `= 0` to track the number of successfully matched words.

S4: Iterate the right pointer `r` by `wordlen`. Extract the current word:
`string word = S.substr(r, wordlen);`

S5: Invalid Word Handling: If the extracted word is NOT in mp1 (! mp1.count(word)), everything collected so far is useless. Clear mp2, reset count = 0, and jump the left pointer strictly past this invalid word: $l = r + \text{wordlen}$.

S6: Valid Word Tracking: If it is a valid word, increment its frequency in mp2 and increment the total count.

S7: Excess Word Handling: It's possible the same valid word appears more times sequentially than required ($\text{while}(\text{mp2}[\text{word}] > \text{mp1}[\text{word}])$). In this case, slide the left pointer l forward by wordlen , removing words from mp2 and decreasing count until the condition is met.

S8: Match Found: If $\text{count} == \text{wordcount}$, a valid substring has been identified. Push the left pointer l to the answer vector. Slide the window forward by dropping the leftmost word to check for overlapping combinations.

Solution.cpp

```
class Solution {
public:
    vector<int> findSubstring(string s, vector<string>& words) {

        int n = s.size();
        int wordcount = words.size();
        int wordlen = words[0].size();
        vector<int> ans;

        unordered_map<string,int> mp1;
        for(string s : words){
            mp1[s]++;
        }

        for(int i = 0; i < wordlen; i++){ //basically ye offset btayega har word pe
            unordered_map<string,int> mp2;
            int l = i;
            int count = 0;

            for(int r = i; r + wordlen <= n; r += wordlen){ //poora poora word check k
                string word = s.substr(r, wordlen);

                if(!mp1.count(word)){ //different word he to abhi tak jo bhi checking
                    mp2.clear();
```

```

        count = 0;
        l = r + wordlen;
        continue;
    }

    mp2[word]++; //otherwise agar seen word he to uska frequency badha do
    count++;

    while(mp2[word] > mp1[word]){ //ho skta he ek hi word baar baar repeat
        string remove = s.substr(l, wordlen);
        mp2[remove]--;
        count--;
        l += wordlen;
    }

    if(count == wordcount) {
        ans.push_back(l);
        string remove = s.substr(l, wordlen);
        mp2[remove]--;
        count--;
        l += wordlen;
    }
}

return ans;
};

```